Phala Network 전체 시스템 아키텍처 분석

1. 시스템 개요

Phala Network는 **TEE 기반 오프체인 컴퓨팅 프로토콜**로, AI 에이전트를 위한 탈중앙화된 실행 레이어를 제공하는 완전한 블록체인 생태계입니다.

2. 전체 아키텍처 구조

```
Phala Network 전체 시스템

├── 1. 블록체인 레이어 (Substrate 기반)

| ├── Phala Node (standalone/node)

| ├── Runtime (standalone/runtime)

| ├── Executor (standalone/executor)

| └── RPC (standalone/rpc)

|

├── 2. TEE 컴퓨팅 레이어

| ├── pRuntime (standalone/pruntime)

| ├── Phactory (crates/phactory)

| ├── Pink Runtime (crates/pink)

| └── SideVM (crates/sidevm)

|
```

```
├── 3. 브리지 레이어
│ ├── Pherry (standalone/pherry)
│ ├── PRB (standalone/prb)
│ └── Prouter (standalone/prouter)
│
├── 4. 팰릿 레이어 (pallets/)
│ ├── Phala Pallets (pallets/phala)
│ └── Offchain Rollup (pallets/offchain-rollup)
│
├── 5. 프론트엔드 레이어 (frontend/)
│ ├── Apps (frontend/apps)
│ └── Packages (frontend/packages)
│
├── 6. 테스트 및 도구
│ ├── E2E Tests (e2e/)
│ ├── Scripts (scripts/)
│ └── Standalone Tools (standalone/)
│
└── 7. 지원 라이브러리 (crates/)
├── 암호화 (phala-crypto)
├── 타입 정의 (phala-types)
├── 네트워킹 (phala-mq, prpc)
```

└── 유틸리티 (기타 크레이트들)

3. 핵심 컴포넌트 역할

3.1 블록체인 레이어

- **Phala Node**: Substrate 기반 블록체인 노드
- **Runtime**: 블록체인 런타임 로직 (WASM)
- **Executor**: 런타임 실행 엔진
- **RPC**: 외부 인터페이스 및 API

3.2 TEE 컴퓨팅 레이어

- **pRuntime**: SGX Enclave 내 실행 환경
- **Phactory**: 핵심 비즈니스 로직 엔진
- **Pink Runtime**: 스마트 컨트랙트 실행 환경
- **SideVM**: 격리된 가상머신 환경

3.3 브리지 레이어

- **Pherry**: 블록체인-TEE 브리지
- **PRB**: Phala Relayer Bridge
- **Prouter**: 프로토콜 라우터

3.4 팰릿 레이어

- **Phala Pallets**: Phala 특화 블록체인 모듈
- **Offchain Rollup**: 오프체인 롤업 구현

4. 데이터 플로우

4.1 사용자 요청 처리 플로우

```
1. 사용자 요청
├── Frontend Apps → RPC API
├── 블록체인 트랜잭션 생성
└── 팰릿에서 처리

2. TEE 실행 요청
├── Pherry가 블록체인 이벤트 감지
├── pRuntime으로 요청 전달
├── Phactory에서 비즈니스 로직 실행
└── Pink Runtime에서 스마트 컨트랙트 실행

3. 결과 반환
├── TEE 실행 결과 생성
├── Pherry를 통해 블록체인에 반영
├── 상태 업데이트
└── 사용자에게 결과 반환
```

4.2 상태 동기화 플로우

```
1. 블록체인 상태 변경
```

```
├── 새 블록 생성
├── 상태 루트 업데이트
└── 이벤트 발생

2. TEE 동기화
├── Pherry가 블록 헤더 수신
├── pRuntime으로 상태 전달
├── Phactory에서 상태 검증
└── 로컬 상태 업데이트

3. 일관성 보장
├── 상태 루트 검증
├── 머클 증명 확인
└── 롤백 처리 (필요시)
```

5. 보안 모델

5.1 다층 보안 구조

```
보안 레이어:
├── L1: 하드웨어 보안 (Intel SGX)
├── L2: 소프트웨어 보안 (pRuntime)
├── L3: 프로토콜 보안 (Phala Pallets)
```

```
├── L4: 네트워크 보안 (P2P, TLS)
└── L5: 애플리케이션 보안 (Smart Contracts)
```

5.2 신뢰 모델

- **하드웨어 신뢰**: Intel SGX 하드웨어
- **코드 신뢰**: 오픈소스 코드 검증
- **네트워크 신뢰**: 탈중앙화된 검증자 네트워크
- **암호학적 신뢰**: 암호학적 증명 및 서명

6. 확장성 설계

6.1 수평 확장

- **워커 클러스터**: 여러 TEE 워커 노드
- **로드 밸런싱**: 요청 분산 처리
- **상태 샤딩**: 상태 데이터 분할 저장

6.2 수직 확장

- **SideVM**: 추가적인 컴퓨팅 리소스
- **Pink Runtime**: 스마트 컨트랙트 병렬 실행
- **배치 처리**: 트랜잭션 배치 처리

7. 개발 및 배포 환경

7.1 개발 도구

```
개발 환경:
├── Rust Toolchain (2021 Edition)
├── Substrate Framework
├── SGX SDK (Intel)
├── Docker (컨테이너화)
└── Node.js (프론트엔드)
```

7.2 빌드 시스템

```
빌드 프로세스:
├── Cargo Workspace (Rust)
├── Substrate WASM Builder
├── SGX Enclave Build
├── Docker Multi-stage Build
└── Frontend Build (Webpack/Vite)
```

8. 모니터링 및 관찰성

8.1 로깅 시스템

- **구조화된 로깅**: tracing 크레이트 사용
- **보안 로깅**: 민감한 정보 필터링
- **분산 로깅**: 여러 컴포넌트 로그 수집

8.2 메트릭 수집

- **성능 메트릭**: CPU, 메모리, 네트워크
- **비즈니스 메트릭**: 트랜잭션 처리량, 응답시간
- **보안 메트릭**: 공격 탐지, 이상 행동

9. 네트워크 토폴로지

9.1 P2P 네트워크

```
네트워크 구조:
├── Validator Nodes (합의 참여)
├── Full Nodes (전체 상태 유지)
├── Light Nodes (헤더만 동기화)
└── TEE Worker Nodes (오프체인 컴퓨팅)
```

9.2 통신 프로토콜

- **libp2p**: P2P 네트워킹
- **GRANDPA**: 합의 프로토콜
- **BABE**: 블록 생성 프로토콜
- **HTTP/WebSocket**: RPC 통신

10. 거버넌스 및 업그레이드

10.1 온체인 거버넌스

- **Democracy Pallet**: 제안 및 투표
- **Council**: 기술 위원회
- **Treasury**: 재무 관리
- **Sudo**: 개발 단계 관리자 권한

10.2 런타임 업그레이드

- **Forkless Upgrade**: 하드포크 없는 업그레이드
- **WASM Runtime**: 동적 런타임 교체
- **Migration**: 상태 마이그레이션
- **Rollback**: 업그레이드 롤백 메커니즘

이 문서는 Phala Network의 전체 시스템 아키텍처를 개괄적으로 다룹니다. 다음 문서에서는 각 컴포넌트의 상세 구현을 분석합니다.